

Analyze the two GenAI programs. The analysis must include:

- a. Name of two GenAI programs
- b. How you accessed them (e.g., Google query, CoPilot on my computer)
- c. Prompt used. (You must use the same prompt for both GenAIs.)
- d. Generated code from both GenAIs.
- e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.
- f. Select one to use as the basis for this assignment and justify why you picked it.
- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Prompt used for both programs:

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

ChatGPT: The first AI I used to generate code was ChatGPT, and I accessed it through the web on my computer.

1. **Correctness** - The ChatGPT program had some issues in terms of correctness. Firstly, the program was missing the final curly bracket that closes off main. Because of this, the program would not compile. I added the curly bracket for the purposes of compiling and

running the program. However, the program only provided the user with one opportunity to make a guess, rather than three. This mistake occurred because of where “return 0” is placed. Despite spacing making it seem like return was outside of the while loop, “return 0” was actually inside the while loop. This occurred because the closing bracket was placed after the return. This caused the while loop to only run once before exiting, rather than three times. Further, the code does nothing to handle cases where the user input is invalid, which would be a nice inclusion to the code. This could include when the user gives a string or numbers not between 1 and 10.

2. **Execution time** - ChatGPT’s code technically only runs through the while loop a single time, but that is because of the flaw within the code. As intended, it was supposed to break the loop if the user guessed the correct number. This is beneficial because it does not run through the system repeatedly when unnecessary. The program used a while loop instead of a for loop, which is slightly worse than the Gemini code, because it has to continuously reference and alter a variable to run.
3. **Space complexity** - ChatGPT’s program contains a finite number of variables and no lists or large data structures. This makes space usage very minimal and appropriate for the program size and function.
4. **Maintainability** - ChatGPT’s program uses many variables, which makes the code easily adaptable. It also helps to avoid missing changes if you change something like the secret number. However, it could have used a variable for max attempts, just in case someone wanted to change the number of guesses given. Further, the ChatGPT code contained no comments whatsoever, which is bad for maintainability.

ChatGPT Code:

```
#include <stdio.h>

int main() {
    int secretNumber = 7;
    int guess;
    int attempt = 1;

    printf("Guess a number between 1 and 10.\n");

    while (attempt <= 3) {
        printf("Attempt %d/3. Enter your guess: ", attempt);
        scanf("%d", &guess);

        if (guess == secretNumber) {
            printf("Correct! You win!\n");
            break;
        }
        else if (guess < secretNumber) {
            printf("Too low! Try again.\n");
        }
    }
}
```

```

    }

    else {
        printf("Too high! Try again.\n");
    }

    attempt++;

if (guess != secretNumber) {
    printf("You lose! The secret number was %d.\n", secretNumber);
}

return 0;
}

```

Gemini: The second AI I used to generate code was Gemini, and I accessed it through the web on my computer.

1. **Correctness** - The code does nothing to handle cases where the user input is invalid, which would be a nice inclusion to the code. This could include when the user gives a string or numbers not between 1 and 10. Other than that, the program functioned as intended.
2. **Execution time** - This program breaks the loop if the user correctly guesses the number, which is good for execution time. It does not loop through extra times unnecessarily. Additionally, this program uses a for-loop, which is better for execution time than a while loop. This is because the program does not need to continuously reference the variable. There is some duplicated logic and loops in this code when printing “too low” and “too high” that could be removed to improve execution time.
3. **Space complexity** - Gemini uses a finite amount of variables and is not incredibly complex space-wise. Additionally, it does not require any arrays or large data structures.
4. **Maintainability** - Gemini uses descriptive variable names and refers to them throughout the code. This is good for maintainability because it makes code more readable and understandable. Further, it makes it easier for users to change variables, such as `max_attempts`, without missing occurrences throughout the code. Without variables, inconsistencies could arise. Further, the section that prints “try again” or “new line” could have been a function to avoid repeated code. Finally, the comments throughout the code are very sparse. The code could use control statements and line comments.

Gemini Code:

```
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int max_attempts = 3;
    int won = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= max_attempts; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt, max_attempts);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;
            break; // Stops the loop immediately on a correct guess
        } else if (guess < secret) {
            printf("Too low!");
            if (attempt < max_attempts) {
                printf(" Try again.\n");
            } else {
                printf("\n");
            }
        } else {
            printf("Too high!");
            if (attempt < max_attempts) {
                printf(" Try again.\n");
            } else {
                printf("\n");
            }
        }
    }

    // Print final lose message if the user didn't win
    if (!won) {
        printf("Out of attempts! The secret number was %d. You lose.\n", secret);
    }
}
```

```
    return 0;  
}
```

f) Code chosen to edit - I am choosing to edit the Gemini program because of correctness, maintainability, and execution time. Firstly, Gemini's program actually runs the full three times. ChatGPT's code cuts the user off after one guess. Additionally, Gemini used a for loop, which is more time-conservative than ChatGPT's while loop. Next, Gemini used a variable for max attempts, while ChatGPT did not, making it slightly more maintainable long term. ChatGPT hardcodes the number 3 a few times throughout the code, instead of using a variable for max attempts. This can make maintainability within the program challenging. Additionally, Gemini included a few comments throughout, which is better for maintainability.

g) What I did to improve the code:

- a. Correctness** - In order to handle edge cases, I added code that would repeatedly loop until the user gives valid input. If the user puts in a string or a number below one or above ten, they are prompted to give the input again. This adds some time complexity and space complexity. However, I believe it is a good tradeoff for more effective functionality.
- b. Execution time** - I removed the repeated print of "try again" and "\n", so that they print from the same spot, instead of checking in multiple different spots. This slightly improves the speed performance of the code.
- c. Space complexity** - I made no meaningful improvement to space complexity because the program was already very minimal. It only contained a finite number of variables used to keep the program maintainable.
- d. Maintainability** - To improve the maintainability of the program, I added comments throughout. I added comments for control structures and each line of code. Additionally, I consolidated duplicated code so that it occurs only once in the program.